

1. What exactly should your project do?

In simple terms:

Take rainfall + weather + terrain + river/soil + historical flood information → calculate the probability of a flash flood for a selected hilly location → show the risk on a map → explain why the risk is high → generate an early warning.

For example:

Location: Dehradun / a selected Himalayan region

Current conditions:

- Rainfall: 85 mm/hour
- Rainfall in previous 3 hours: 150 mm
- Soil moisture: High
- Slope: 32°
- Elevation: 1,450 m
- Distance from river: 400 m
- Forecast rainfall: Heavy

↓

ML model:

Flood probability = 87%

↓

System:

CRITICAL FLOOD RISK

↓

Map:

- High-risk areas
- Medium-risk areas
- Low-risk areas

↓

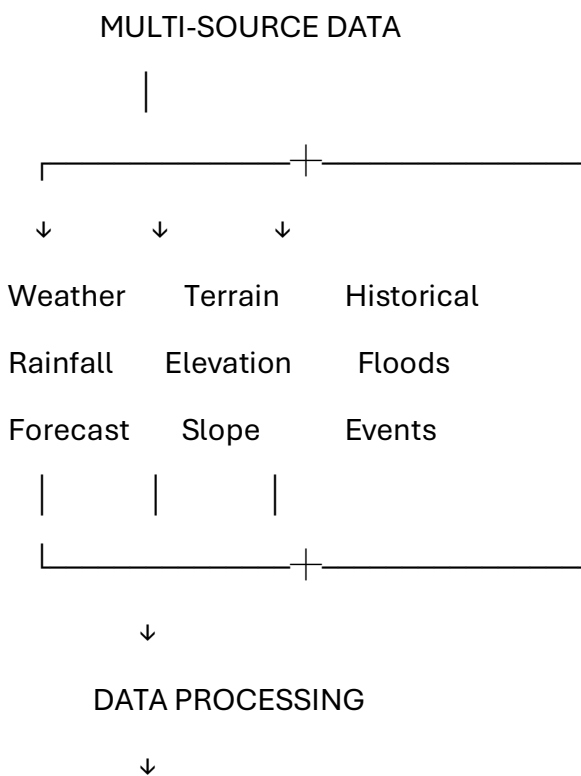
Warning:

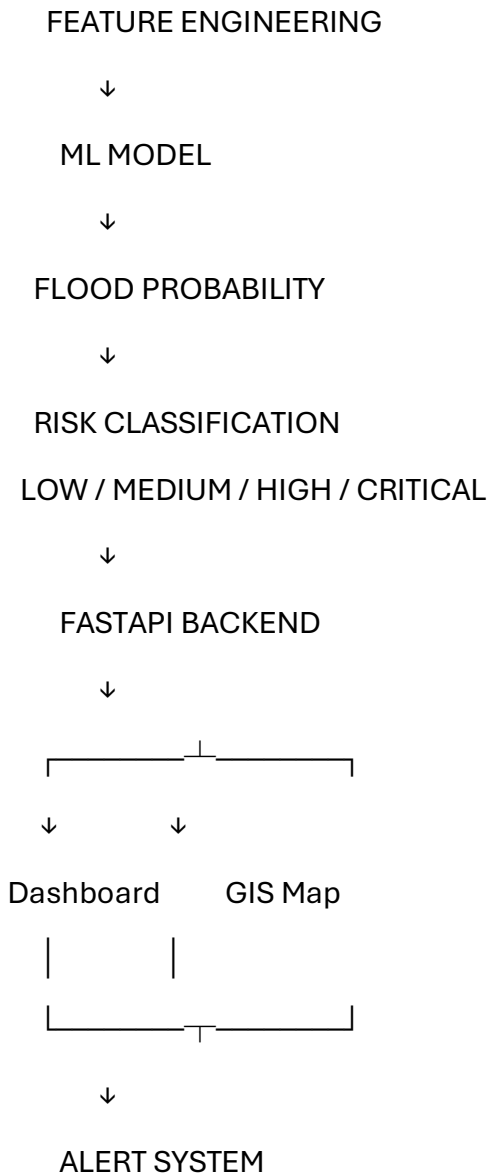
⚠ Flash flood risk is HIGH. Heavy rainfall combined with saturated soil and steep terrain may cause rapid runoff. Avoid low-lying areas and river channels.

That's your core project.

2. Complete workflow

Your system can be thought of as this:





3. Technology stack — keep it simple

I strongly recommend this stack:

Technology	Purpose	Why
Python	ML + data processing	Easy and powerful
Pandas	Dataset processing	Simple tabular data handling

NumPy	Numerical operations	ML/data calculations
Scikit-learn	ML	Easy model training/evaluation
XGBoost	Main ML model	Excellent for tabular data
FastAPI	Backend/API	Very simple Python backend
HTML/CSS/JavaScript	Frontend	You already have web-dev foundation
Leaflet.js	Interactive map	Easy GIS visualization
PostgreSQL + PostGIS	Database/geospatial data	Good but still manageable
Git + GitHub	Team collaboration	Essential

4. What data do you actually need?

Think about the physical question:

What conditions make a hilly area more likely to experience flash flooding?

You need variables representing those conditions.

Weather/rainfall

Important:

- Current rainfall
- Rainfall last 1 hour
- Rainfall last 3 hours
- Rainfall last 6 hours
- Rainfall last 24 hours
- Forecast rainfall
- Temperature
- Humidity

Terrain

From DEM/elevation data:

- Elevation
- Slope
- Aspect
- Terrain characteristics

Hydrology

Useful:

- Distance from river
- River level
- Drainage characteristics
- Accumulated rainfall

Soil/environment

Potentially:

- Soil moisture
- Soil type
- Land cover

Historical information

Most important for ML:

- Did flooding occur?
- When?
- Where?
- What rainfall conditions existed?

5. Where can you obtain the data?

For a hackathon, don't try to collect everything yourself.

Use reliable public sources such as:

Rainfall/weather

Possible sources include:

- IMD
- NASA GPM
- ERA5
- Open-Meteo
- government weather datasets

For your prototype, **historical rainfall data + a weather API** is enough to start.

Elevation

Use:

- SRTM
- NASA DEM
- other publicly available DEM datasets

From elevation you can derive:

Elevation → Slope
→ Terrain characteristics

Satellite

Potentially use:

- Sentinel
- Landsat

But satellite imagery should be **advanced**, not your first task.

Flood history

Look for:

- historical flood-event datasets
- disaster databases
- government reports
- rainfall/flood records

The key is that you eventually want something like:

Location + Date/Time + Weather/Terrain conditions → Flood/No Flood

6. What should your ML dataset look like?

Imagine your final training dataset looks like:

rainfall _1h	rainfall _3h	rainfall_ 24h	humid ity	elevati on	slo pe	soil_mois ture	river_dista nce	floo d
12	35	72	82	1450	28	0.62	500	0
65	142	280	94	1200	35	0.91	300	1
8	21	50	70	1600	20	0.40	900	0
82	190	350	97	1100	38	0.95	250	1

The important concept is:

INPUT FEATURES

↓

ML MODEL

↓

FLOOD = 0 or 1

Later, instead of only predicting 0/1, you can produce:

Flood probability = 0.87

7. How do different datasets get combined?

This is one of the most important parts of your project.

Suppose you have:

Weather data

timestamp | rainfall | humidity

Terrain data

latitude | longitude | elevation | slope

Flood data

latitude | longitude | timestamp | flood

You need to bring them onto a common:

location + time

framework.

Conceptually:

Location A

+

Rainfall at Location A

+

Elevation at Location A

+

Slope at Location A

+

Soil moisture at Location A

+

Historical flood status



ONE TRAINING ROW

This process is called **data integration/feature engineering**.

Don't worry about building this perfectly on Day 1. First get a small working dataset.

8. Which ML model should you use?

Start with Random Forest

It's extremely beginner-friendly.

Then try:

XGBoost

I would ultimately make **XGBoost your main model** if your data supports it.

Comparison

Model	Recommendation
Logistic Regression	Baseline
Random Forest	★ Start here
XGBoost	★ ★ ★ Main model
Neural Network	✗ Not necessary
CNN/LSTM	✗ Don't start here

Your data is primarily **structured/tabular data**.

You don't need deep learning just because the project contains "AI."

9. What is the ML target?

You have two good possibilities.

Option A — Classification

0 = No flood

1 = Flood

Then model outputs:

$P(\text{flood}) = 0.87$

Convert it to:

0-30% → LOW

30-60% → MODERATE

60-80% → HIGH

80-100% → CRITICAL

This is excellent for your MVP.

Option B — Flood severity

Later:

Low

Moderate

High

Severe

But don't start there.

Start with binary flood prediction + probability.

10. How should you evaluate the model?

Don't tell judges:

"Our accuracy is 98%."

That can easily be misleading with flood datasets.

Show:

- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion matrix

For flood prediction, **Recall is particularly important** because missing an actual flood event is potentially much worse than generating a false warning.

Also make sure your train/test split respects **time and location**.

For example, don't randomly mix observations from the same flood event into both training and testing.

Otherwise:

Same flood event

↓

training data

+

test data

can make your model appear much better than it really is.

11. Complete development roadmap

Now let's turn this into an actual project plan.

PHASE 1 — DATA

Build

Start by obtaining a small dataset containing:

```
rainfall  
elevation  
slope  
soil moisture  
river distance  
historical flood label
```

Technology

Python + Pandas

Output

Something like:

```
flood_training.csv
```

Before moving on

You should be able to answer:

"What does one row of our dataset represent?"

PHASE 2 — ML MODEL

Build

Create:

data
↓
preprocessing
↓
train/test split
↓
Random Forest
↓
evaluation
↓
XGBoost
↓
probability

Output

A trained model:

flood_model.pkl

And a function conceptually like:

Input:

rainfall = 80

slope = 35

soil_moisture = 0.9

...

Output:

probability = 0.87

risk = CRITICAL

Before moving on

You need a model that works on test data.

PHASE 3 — BACKEND

Build an API using **FastAPI**.

Your frontend sends:

```
{
  "latitude": 30.3165,
  "longitude": 78.0322,
  "rainfall_1h": 80,
  "rainfall_3h": 150,
  "slope": 35,
  "soil_moisture": 0.9
}
```

Backend sends:

```
{
  "flood_probability": 0.87,
  "risk_level": "CRITICAL"
}
```

Output

Working API.

PHASE 4 — GIS / MAP

Use:

Leaflet.js

Display:

MAP

● Critical

● High

● Low

● Moderate

You can use latitude/longitude points or polygons.

Later you can create a **risk heatmap**.

Important

You don't need to build a GIS engine from scratch.

Leaflet does the visualization.

PHASE 5 — FRONTEND

Create a dashboard.

Something like:

FLASH FLOOD EARLY WARNING SYSTEM

Location: [Dehradun ▼]

Current Rainfall:	82 mm
Rainfall 3h:	160 mm
Soil Moisture:	91%
Elevation:	1450 m
Slope:	34°

FLOOD RISK

87%
CRITICAL

MAP

Risk Factors:

- ✓ Extreme rainfall
- ✓ High soil moisture
- ✓ Steep slope
- ✓ Close to river

⚠ EARLY WARNING GENERATED

That's already a strong hackathon demo.

PHASE 6 — REAL-TIME / FORECAST DATA

Once your basic system works, connect APIs.

For example:

```
Weather API
  ↓
Current rainfall/weather
  ↓
FastAPI
  ↓
ML model
  ↓
Risk
```

For the hackathon, if an API doesn't provide exactly the historical variable you need, **don't get stuck.**

You can use:

Historical data → ML training

Live/forecast data → demonstration

PHASE 7 — ALERTS

Create simple threshold logic:

Probability < 30%



LOW

30–60%



MODERATE

60–80%



HIGH

>80%



CRITICAL



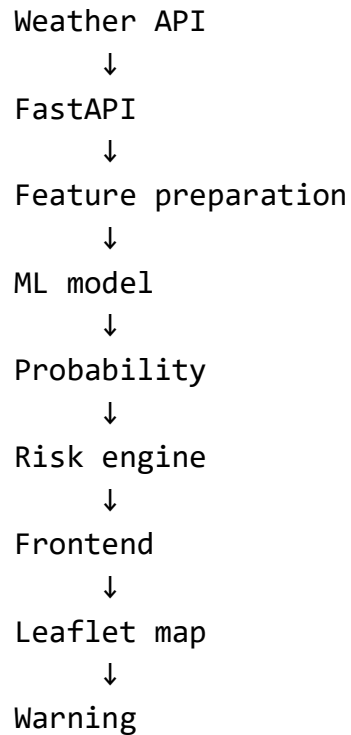
WARNING

For the prototype, an in-dashboard warning is sufficient.

You can optionally add email/SMS/notification later.

PHASE 8 — INTEGRATION

This is where everything becomes **one project**.

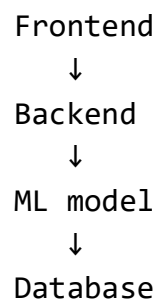


Test the complete flow repeatedly.

PHASE 9 — DEPLOYMENT

Only after everything works locally.

A simple architecture is enough:



You can deploy later using a simple cloud platform.

Don't spend the first week learning cloud deployment.

